PHP end-of-term exam RETAKE (2021-22-1)

Due No Due Date **Points** None

Important information

- You have **150 minutes** to complete the exam. We provide you **additional 30 minutes** for filling in the `README.md` (see template below), reading the problems, downloading the materials, compressing and uploading your solution.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 19:00 (Hungarian time) and there will be no opportunity to submit your solution after the aforementioned timepoint.**
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)!
- The problems are independent and can be solved **in any order**.
- To begin your work you should first **download the starting package we have prepared**. Each tasks has its own directory in the package. All tasks are somewhat prepared. Other than the provided files, you are free to use any other files (to store data, organize your code, implement AJAX/fetch), but make sure your solutions are in the correct folder and can be started with the `index.php` file!
- The starting package contains a template called `README.md` where you should fill in your name and Neptun code (the <> characters must be removed). **Submissions without a properly filled `README.md` file will not be graded!**
- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

1. task: Bike sharing (1-bicycles, 8 points)

Today, more and more people are paying attention to the environment and sustainable development, making the use of bicycles a popular alternative to car and public transport. Several cities in Hungary already have some form of bicycle sharing system. We have the data of the collection stations of such a system operating in Budapest. The `$stations` array included from the `data.php` file contains the year each station was opened and the number of bike docks in it, while the `$locations` array specifies the station address and district number, and contains the station identifier. **Before starting the coding, it is worth studying and understanding the structure of the given data!**

*Technical help: you can extract column information from an array of records with the **array_column() (https://www.php.net/manual/en/function.array-column.php)** function. Map and filter programming patterns can be easily carried out with the **array_map() (https://www.php.net/manual/en/function.array-map.php)** and **array_filter() (https://www.php.net/manual/en/function.array-filter.php)** functions. The **array_unique() (https://www.php.net/manual/en/function.array-unique.php)** method can be used to select individual values in an array.*

- a. (1 point) Generate a table with the first column showing the fixed values of the districts of Budapest from *Budapest 1* to *Budapest 14*!
- b. (2 points) In the second column of the table, indicate how many **different** stations there are in the given district! (Sometimes the same station is identified by more than one address because, for example, it is located at an intersection or at a prominent location. You must specify how many **different** `station_id` values there are in the `$locations` array next to a specific `district` value.)
- c. (2 points) In the third column of the table, indicate in which year the first station in the given district opened!
- d. (1 point) In the fourth column of the table, mark with a separate 🏆 icon the districts, where the station was opened in the year when the bike sharing system started (this is the smallest number of years in the data)!
- e. (2 points) In the last column of the table, enter the average number of bicycle docks per station in the given district.

RENTABLE BICYCLES

DISTRICT	STATIONS	OPENED	🏆	AVG DOCKS
Budapest 1	4	2015		18.45
Budapest 2	14	2012	🏆	25.05
Budapest 3	7	2014		21.34
Budapest 4	7	2012	🏆	25.50
Budapest 5	10	2013		12.40
Budapest 6	11	2012	🏆	18.64
Budapest 7	12	2013		18.66
Budapest 8	14	2012	🏆	22.52
Budapest 9	11	2013		22.22
Budapest 10	3	2019		20.00
Budapest 11	19	2012	🏆	26.61
Budapest 12	8	2015		21.85
Budapest 13	15	2016		20.65
Budapest 14	2	2012	🏆	25.00

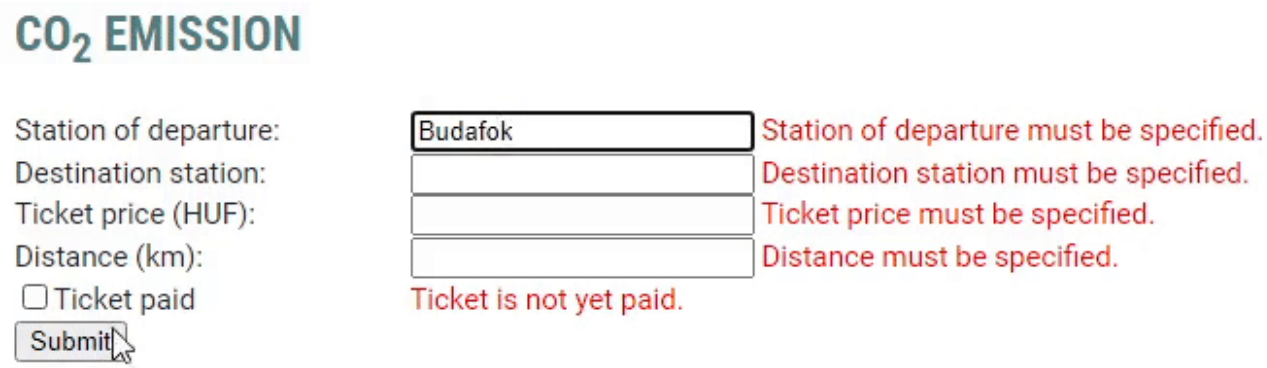
2. task: CO_2 emission (2-co2, 8 points)

The *Hungarian State Railways Company* also knows exactly that one of the biggest challenges of the 21st century is to align mobility needs with environmental protection. Anyone, who has recently chosen the railway, could also see on the ticket how much of the pollutant emissions (mainly carbon dioxide) was saved from the environment, compared to private car transport. Create a page that validates the data received as a `GET` parameter for a trip and calculates the CO_2 savings if the data is correct! **In the case of an invalid field, display an error message that clearly indicates which condition the sent value did not meet!**

- a. (1 point) The `journey_from` (departure station) parameter is mandatory and contains a value that is included in the pre-defined `$machines` array (names of stations with ticket machines).
- b. (1 point) The `journey_to` (destination) parameter is mandatory and cannot be the same as the

departure station.

- c. (1 point) The `price` (ticket price) parameter is mandatory, non-negative integer *(can be 0Ft)*.
- d. (1 point) The parameter `distance` is mandatory, positive float *(cannot be 0.00 km)*.
- e. (1 point) The `paid` box must be checked!
- f. (1 point) The form on the page should keep its state, so the values sent should be written back in the input fields!
- g. (1 point) In case of an incorrect input, the error message should be displayed in red next to the incorrect field!
- h. (1 point) If all parameters meet the above conditions, the page should show the reduction in CO_2 emissions compared to individual car transport, which is 0.13 kg per kilometer traveled!



3. task: Reuse advertisments (3-reuse, 8 points)

Reuse also plays a major role in the production of less waste. Create a mini advertisment page where we can advertise things that others can reuse! In the page, you can list ads, search for them, post a new one, view the details of an ad, and take an advertised item. The data is stored in the `ads.json` JSON file, which also contains some example data. For this solution, it is recommended to use the Storage class introduced in the lectures.

- a. (1 point) On the `index.php` page, list the ads in the JSON file according to the given example.
- b. (1 point) On the `index.php` page, list only ads with `active` value in the `status` property!
- c. (2 points) At the top of the `index.php` page there is a form for creating a new ad. By submitting the form, save the new ad and list it! When storing, in addition to the `title`, `description`, `email` and `url` data, store also the `status` with the value of `active` and the date of creation in the `created_at` field in the format of `YYYY.mm.dd. hh:mm:ss`. For the latter, use the format string `"Y.m.d. G:i:s"` in PHP's `date()` function (https://www.php.net/manual/en/function.date.php). After saving, redirect browser to `index.php`!
- d. (1 point) In the ad list, create a link from the ad title and refer to `details.php` by passing the ID of the current ad, e.g. `details.php?id=1`.
- e. (1 point) On the `details.php` page, display the details of the ad that matches the ID that arrives as the

GET parameter.

- f. (1 point) On the `details.php` page update your ad by submitting an email address as follows:
  - save the incoming email address as `email2` for the ad;
  - set the `status` of the ad to `inactive`!

  After a successful save, redirect your browser to `index.php`.
- g. (1 point) By filling out the filter field on the `index.php` page and submitting the form, display only those ads that are active and contain the filter text in either their title or description!

REUSE ADVERTISMENTS

NEW AD

Title:
Wooden cutlery

Description:
For eating

Contact email:

Image URL:

Add

ACTIVE ADS

Filter

Filter text:

Search



Reusable bag

A bag for buying things

2022.01.13. 12:21:20



Reusable plastic box

Never used

2022.01.13. 12:22:59

4. task: Survey on recycle (4-survey, 8 points)

"Not all waste is rubbish!" In order to protect the environment, it is important that we separate our waste

properly and recycle as much as possible! The task will be to create a questionnaire in which you need to specify in a few steps how much of the given type of garbage you produce per week. The survey is "wizard-like", meaning that each question will appear on a separate page, and you will see a summary page at the end. The data should be stored **in the session**!

- a. (1 point) When calling the `index.php` file, the current page is indicated in a `page` parameter, e.g. `index.php?page=3`. Display the current page number in the header. If there is no `page` parameter, the current page should be `1`! The questions for each page are listed in the `$questions` array. Display the specific question on each page!
- b. (1 point) For navigation links, the "Previous page" link should point to the previous page, but should not appear on page 1! The "Next page" link should point to the next page, but should not appear on page 5!
- c. (1 point) Clicking on the "Save" button on the given page, save the value of the answer for the given page in the session!
- d. (1 point) If there is already an answer for that page, display it in the input field for the answer.
- e. (1 point) The "Next page" link should only be displayed if there is already an answer for that page! If there is an answer to the last page, replace the "Next page" with the "Results" link pointing to `eval.php`!
- f. (1 point) On `eval.php`, show how much communal waste produced per week compared to the weekly recycled waste (answer 1 vs answers 2 + 3 + 4 + 5).
- g. (2 points) On the `eval.php` page, click the "Refresh" button to update the values specified in the previous subtask with AJAX! That is, if we refresh one of the answers on the `index.php` page, we will see the new results by clicking on "Refresh" on the `eval.php` page without reloading the whole page!

